**Institut Teknologi Bandung**

Directorate of Continuing Professional Education

in partnership with

CHAPTER 14

# Text Classification

Tokenization, and the one question every NLP architecture answers differently: is a sentence a set of words, or a sequence of them?

**Duration** 3 hours (2 sessions) **Instructor** Rahman Indra Kesuma, S.Kom., M.Cs.

**Source** Chollet & Watson, Deep Learning with Python 3e — chapter 14

LEARNING OUTCOMES

# Session Objectives

By the end of this session, participants can:

1. Explain why natural language is **messy in a way machine languages are not**, and why rule-based NLP failed.
2. Compare **character, word, and subword** tokenization, and say what byte-pair encoding optimises.
3. State the **set versus sequence** question and place bag-of-words, RNNs, and Transformers against it.
4. Build a **bag-of-words** classifier with `TextVectorization`, and extend it with **bigrams**.
5. Explain what a **word embedding** is, and why one-hot encoding makes a false assumption.
6. **Pretrain** an embedding with an unsupervised task (CBOW), and use it for classification.

BOOK
Chapter 14 — full text

NOTEBOOK
01_tokenizers.ipynb

NOTEBOOK
02_bag_of_words_and_bigrams.ipynb

NOTEBOOK
03_sequence_models.ipynb

NOTEBOOK
04_pretraining_an_embedding.ipynb

REPO
Author's official notebook

SECTION 14.1

# Why "natural" language is a different kind of object

**Machine languages were designed.** An engineer wrote down formal rules first; people used the language once the rule set was complete.

**Human language is the reverse.** Usage comes first; rules arise later, are formalised after the fact, and are **often ignored or broken by its users**

So machine-readable language is highly structured and rigorous, while natural language is **messy — ambiguous, chaotic, sprawling, and constantly in flux**.

SECTION 14.1

# The 1954 demo, and the decade it bought

The goal was to attract excitement and funding, and in that sense **it was a huge success**. The authors claimed translation would be solved within five years. **Funding poured in for the better part of a decade.**

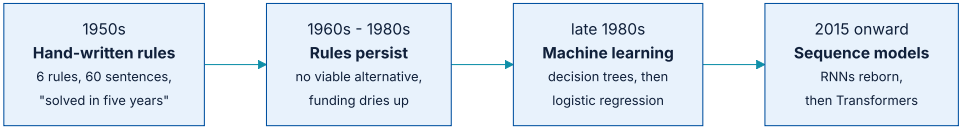
SECTION 14.1

# . . . and why it did not generalise

- **Words change meaning dramatically depending on context.**
- **Every grammar rule needed countless exceptions.**
- Shining on a few handpicked examples was simple; building a robust system that could compete with human translators **was another matter**.

An influential US report a decade later picked apart the lack of progress, and the funding dried up. Yet handcrafted rules **remained dominant well into the 1990s** — because there was no viable alternative.

# The moment it became machine learning




Faster computers and more data in the late 1980s changed what was possible.

> *When you find yourself building systems that are big piles of ad hoc rules … you're likely to start asking, "Could I use a corpus of data to automate the process of finding these rules?" And just like that, you've graduated to doing machine learning.*
>
> Chollet & Watson, section 14.1

# 01

# Preparing text data

Tokenization: from characters to byte-pair encoding.

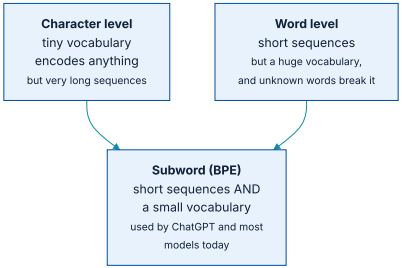
# The simplest tokenizer, written out

listing 14.1 — a character-level tokenizer PYTHON

```python
class CharTokenizer:
    def __init__(self, vocabulary):
        self.vocabulary = vocabulary
        self.unk_id = vocabulary["[UNK]"]

    def standardize(self, inputs):
        return inputs.lower()

    def split(self, inputs):
        return re.findall(r".", inputs)

    def index(self, tokens):
        return [self.vocabulary.get(t, self.unk_id) for t in tokens]

    def __call__(self, inputs):
        return self.index(self.split(self.standardize(inputs)))
```

The `[UNK]` fallback is the important detail: **anything outside the vocabulary becomes one shared token**, and how often that happens is a property of the tokenizer's design.

# Three levels, and the trade-off between them




Subword tokenization aims at the best of both.

SECTION 14.2.2

# Tokenization as compression

- **Shorter tokens** compress the overall length of each example.
- **A smaller vocabulary** reduces the bytes needed to represent each token.
- Achieve both and you feed the model **short, information-rich sequences**.

> That analogy turned out to be powerful: one of the most effective tricks of the last decade was **repurposing a 1990s lossless compression algorithm — byte-pair encoding — for tokenization**. It is used by **ChatGPT and many other models to this day**

SECTION 14.2.2

# How byte-pair encoding builds its vocabulary

the BPE idea, in outline

PYTHON

```
# start: every character is a token
#   l o w e r   n e w e s t   w i d e s t

# merge the most frequent pair, repeatedly:
#   "e" + "s" -> "es"        n e w es t   w i d es t
#   "es" + "t" -> "est"      n e w est    w i d est
#   "l" + "o"  -> "lo"       lo w e r

# stop when the vocabulary reaches the target size
```

Common words end up as **single tokens**; rare words decompose into **familiar pieces**. Nothing is ever `[UNK]`, because **the character level is always available as a fallback**

02

# Sets versus sequences

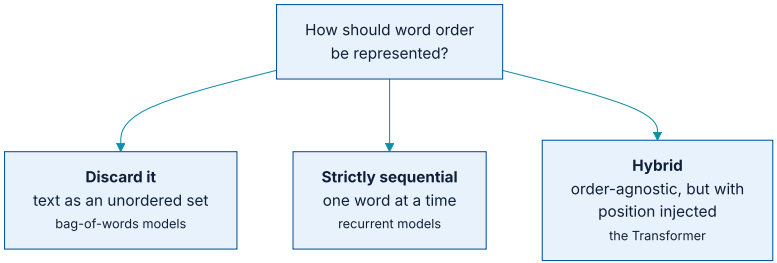
The pivotal question from which every NLP architecture springs.

SECTION 14.3

# Representing a token is easy. Representing order is not.

Unlike the steps of a timeseries, **words in a sentence have no natural, canonical order**. Different languages order similar words very differently; within one language you can usually say the same thing by reshuffling. **Order clearly matters, but its relationship to meaning is not straightforward.**

# Three answers, three families of architecture




RNNs and Transformers both take order into account, so both are called sequence models.

Historically most early NLP was bag-of-words. Interest in sequence models only rose in **2015**, with the rebirth of RNNs. **Both approaches remain relevant today.**

# The benchmark: raw IMDB this time

listing 14.8 — downloading raw IMDB

PYTHON

```python
zip_path = keras.utils.get_file(
    origin="https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz",
    fname="imdb",
    extract=True,
)
imdb_extract_dir = pathlib.Path(zip_path) / "aclImdb"

for path in imdb_extract_dir.glob("*/*"):
    if path.is_dir():
        print(path)
```

OUTPUT

```
aclImdb/train/pos
aclImdb/train/neg
aclImdb/train/unsup
aclImdb/test/pos
aclImdb/test/neg
```

Note `train/unsup` — 25,000 **unlabelled** reviews. They are ignored at first and become essential in section 14.5.4.

# 03

## Set models

Throw order away, and see how far that gets you.

SECTION 14.4

# Bag of words, in one picture


Tokenize, discard order, multi-hot encode.

The idea is simply to **assign a weight to every word**. *terrible* probably indicates a bad review; *riveting* probably indicates a good one.

# TextVectorization does all of it

listing 14.12 — bag-of-words encoding PYTHON

```python
from keras import layers

max_tokens = 20_000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    split="whitespace",         # word-level vocabulary
    output_mode="multi_hot",    # and multi-hot output, in one layer
)

train_ds_no_labels = train_ds.map(lambda x, y: x)
text_vectorization.adapt(train_ds_no_labels)
```

**20,000 words is a good starting point** for text classification. Note `adapt()` is given the reviews **without labels** — it is learning vocabulary, **not learning the task**

SECTION 14.4.1

# The model on top is tiny

the bag-of-words classifier

PYTHON

```python
inputs = keras.Input(shape=(max_tokens,))
x = layers.Dense(16, activation="relu")(inputs)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

It performs respectably — which is the point. **A model that cannot tell *"good, not bad"* from *"bad, not good"* still does well on sentiment**, because long reviews carry plenty of unordered evidence.

SECTION 14.4.2

# Bigrams: putting a little order back by hand

listing — bigram encoding PYTHON

```python
text_vectorization = layers.TextVectorization(
    ngrams=2,                    # unigrams AND bigrams
    max_tokens=max_tokens,
    output_mode="multi_hot",
)
```

One argument, and the model improves: *"not good"* is now a token in its own right. But the approach **only scales to a local ordering of a few words**, and it is feature engineering done by hand.

# 04

# Sequence models

Stop engineering order. Show the model the raw sequence.

SECTION 14.5

# The argument for moving on

As so often in deep learning: rather than building the features yourself, **expose the model to the raw word sequence and let it learn the positional dependencies** — **the same argument chapter 1 made about feature engineering in general**

# One wrinkle: batches have to be rectangular

padding and truncation PYTHON

```python
max_length = 600

text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",           # integer token IDs, in order
    output_sequence_length=max_length,   # pad or truncate to a fixed length
)
```

Padding introduces a new problem the model has to be told about: **those zeros are not words**. Getting this wrong means **the model spends capacity learning about padding**

SECTION 14.5.1

# A recurrent model on the raw sequence

a sequence model

PYTHON

```
inputs = keras.Input(shape=(max_length,), dtype="int32")
x = layers.Embedding(input_dim=max_tokens, output_dim=256, mask_zero=True)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
```

`mask_zero=True` is the answer to the padding problem: it tells every downstream layer to **skip the padded positions entirely**

# 05

# Word embeddings

One-hot encoding makes an assumption that is plainly false.

SECTION 14.5.2

# What one-hot encoding quietly assumes

For words that assumption is **clearly wrong**. *"Movie"* and *"film"* are interchangeable in most sentences, so their vectors **should be the same vector, or close to it** — not at right angles.

SECTION 14.5.2

# What an embedding is instead


Figure 14.3 — sparse, hardcoded, high-dimensional versus dense, learned, and lower-dimensional.

The goal: **the geometric relationship between two word vectors should reflect the semantic relationship between the words**. Related words close, unrelated words far apart.

SECTION 14.5.2

# Directions in the space mean things

- The same vector takes you from **cat** → **tiger** and from **dog** → **wolf** — a *from pet to wild animal* direction.
- Another takes you from **dog** → **cat** and **wolf** → **tiger** — a *from canine to feline* direction.
- In real embedding spaces the classic examples are **gender** and **plural** vectors: add *female* to *king* and you get ***queen***.

> Real word-embedding spaces typically feature **thousands** of such interpretable directions.

SECTION 14.5.3

# Is there one ideal embedding space?

> The book's answer is careful: **possibly — but we have yet to compute anything of the sort.** What is available in practice is an embedding learned either **jointly with your task, or on a large corpus beforehand**

An `Embedding` layer trained with the classifier learns a space specialised to *this* task. That is often fine — and when labelled data is scarce, it is not.

06

# Pretraining

Where the unlabelled 25,000 reviews finally earn their place.

SECTION 14.5.4

# Why pretraining took over NLP

The idea: **devise an unsupervised task that needs no labels**. Pretraining data can be text from a similar domain, or even arbitrary text in the right language. It learns general patterns, **priming the model before you specialise it**

# CBOW: guess the word from its neighbours

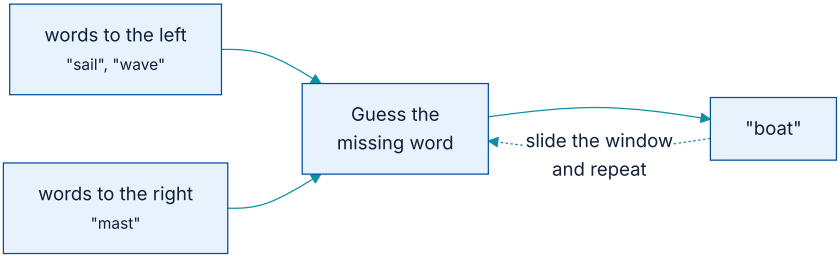



Figure 14.6 — Continuous Bag of Words, from Mikolov et al. (2013).

If the surrounding bag contains *sail*, *wave*, and *mast*, you might guess *boat*. **No labels are needed** — the supervision comes from the text itself, which is exactly chapter 1's **self-supervised learning**

SECTION 14.5.4

# Where the extra data comes from

combining labelled and unlabelled text

PYTHON

```python
# all of train/pos, train/neg AND train/unsup, labels discarded
pretrain_ds = keras.utils.text_dataset_from_directory(
    imdb_extract_dir / "train",
    labels=None,
    batch_size=batch_size,
)
```

**Doubling the text costs nothing**, because the CBOW task does not care whether a review is positive or negative — **it only needs words in context**

SECTION 14.5.5

# Using the pretrained embedding

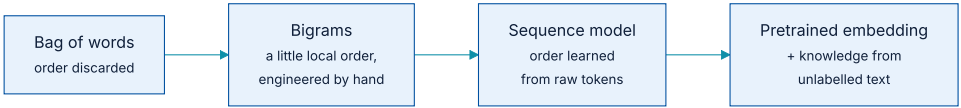
loading pretrained weights

PYTHON

```python
embedding_layer = layers.Embedding(max_tokens, 256, mask_zero=True)
embedding_layer.build((None,))
embedding_layer.set_weights(pretrained_embedding.get_weights())

inputs = keras.Input(shape=(max_length,), dtype="int32")
x = embedding_layer(inputs)
x = layers.Bidirectional(layers.LSTM(32))(x)
outputs = layers.Dense(1, activation="sigmoid")(layers.Dropout(0.5)(x))
```

This is **transfer learning for text**, structurally identical to feature extraction from a pretrained ConvNet — **and it is the pattern the whole of chapter 15 builds on**

# The four models, on one benchmark




Each step adds either more order information or more outside knowledge.

> And the practical caveat: **the bag-of-words model is close behind, trains in seconds, and is trivial to explain**. On short texts, or with little data, **it is frequently the right answer**

SUMMARY

# What has to survive this chapter

1. **Natural language is messy by construction** — usage precedes rules, which is why rule-based NLP failed for forty years.
2. **Tokenization is compression.** Byte-pair encoding gets short sequences and a small vocabulary at once.
3. **Set or sequence** is the question every NLP architecture answers.
4. **Bag-of-words is a strong, cheap baseline.** Bigrams add local order for one argument.
5. **One-hot assumes words are independent**, which is false. Embeddings learn a space where geometry reflects meaning.
6. **Pretraining on unlabelled text** removes the labelled-data bottleneck.

NOTEBOOK

04_pretraining_an_embedding.ipynb

NEXT

Chapter 15 — Language models and the Transformer